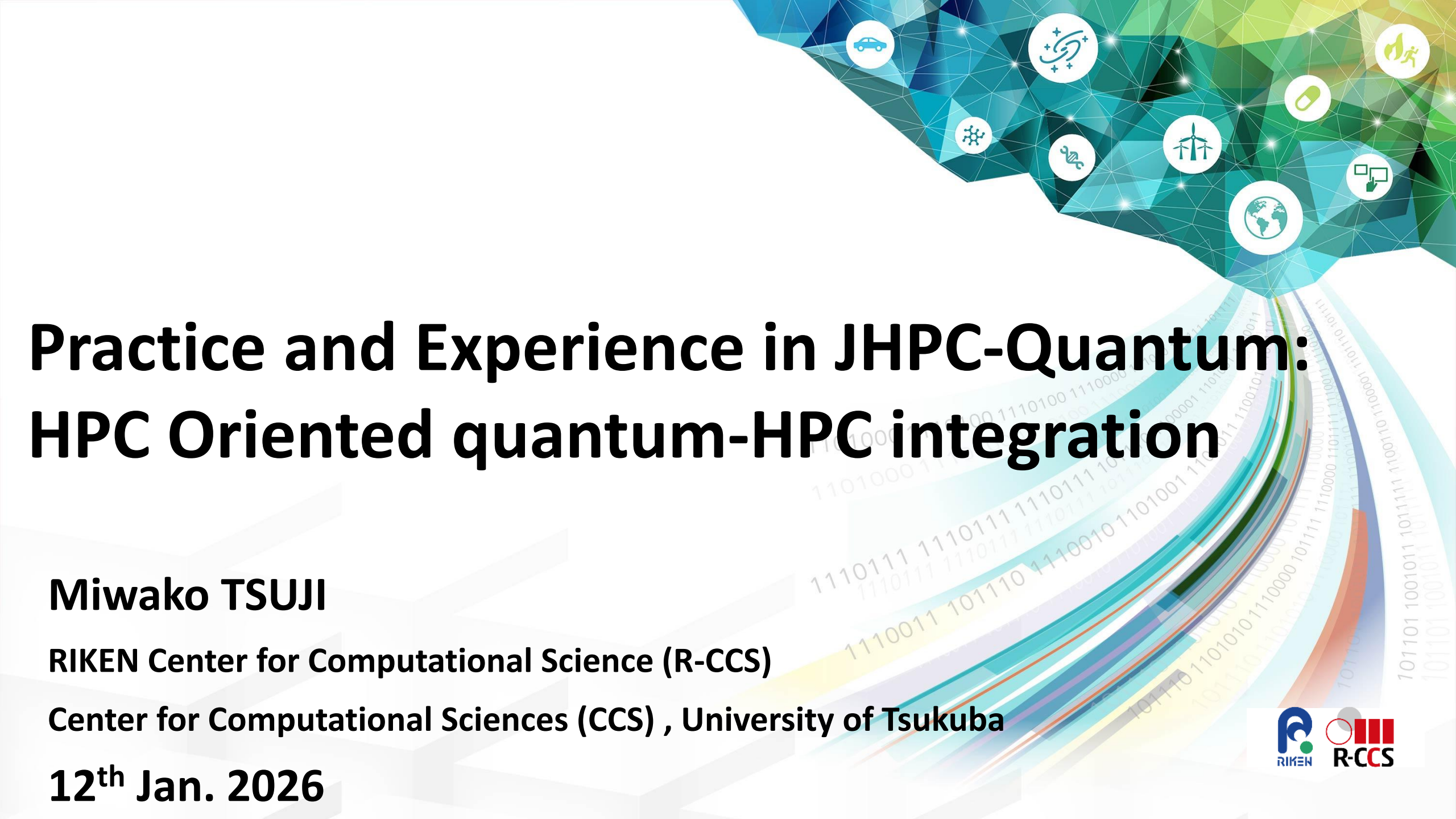




Practice and Experience in JHPC-Quantum: HPC Oriented quantum-HPC integration

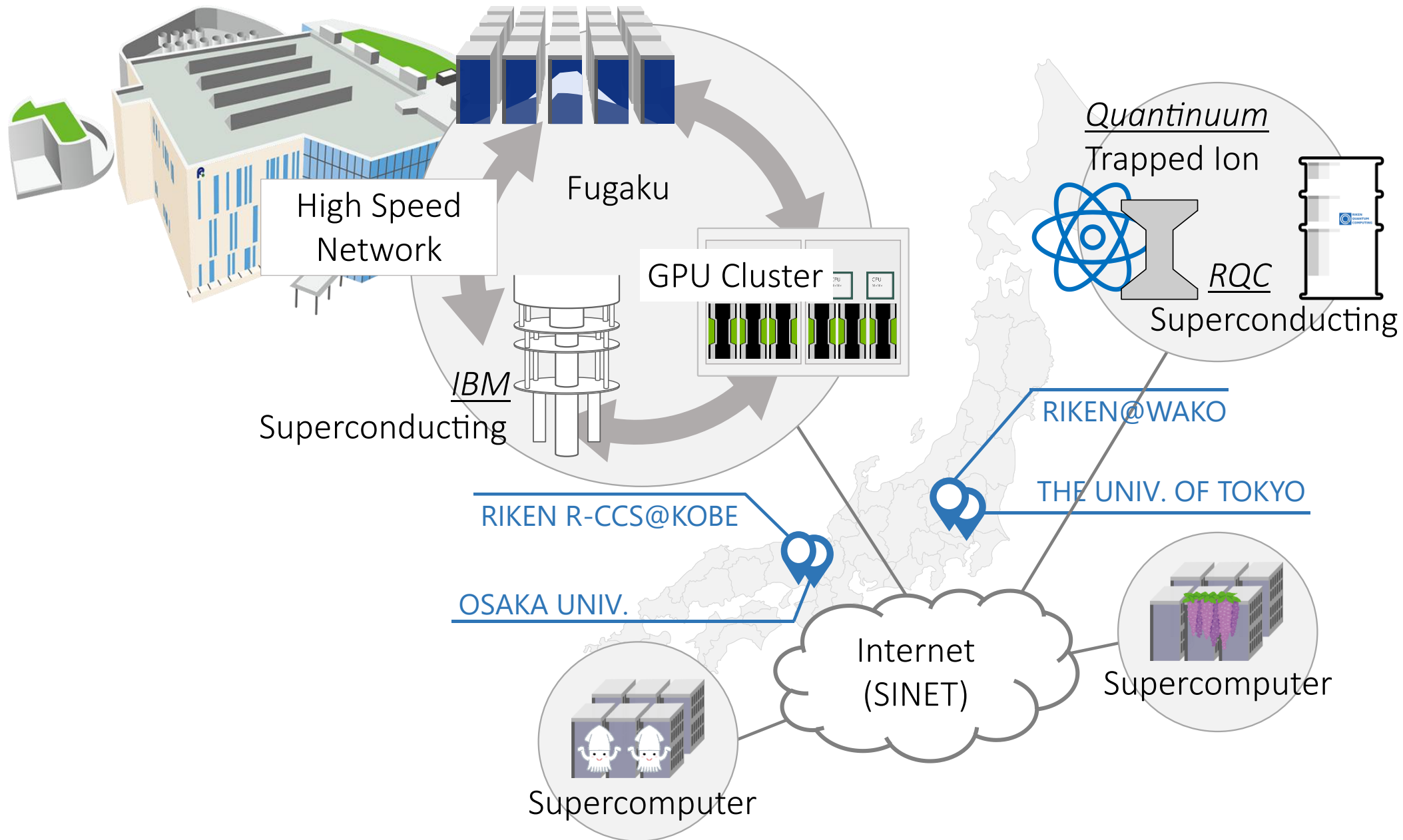
Miwako TSUJI

RIKEN Center for Computational Science (R-CCS)

Center for Computational Sciences (CCS) , University of Tsukuba

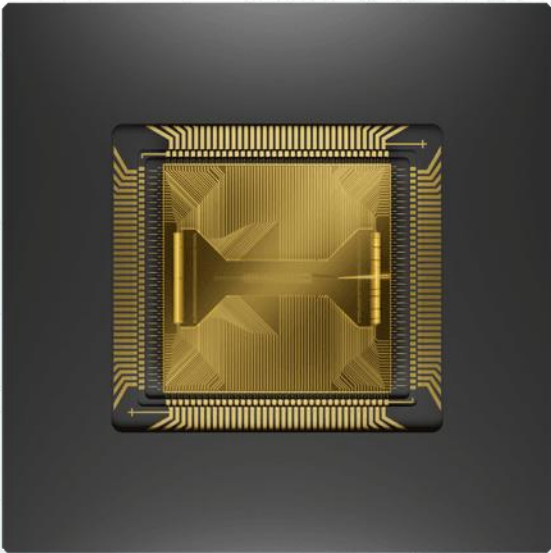
12th Jan. 2026

Overview (JHPC-Quantum & TRIP)



Recent Status of Quantum Systems

	IBM Kobe	Quantinuum 黎明 Reimei	RIKEN 叡 A
Name	Heron	System Model H1	A
Type	Superconductive	Trapped Ion	Superconductive
Qubit	156	20	64
Status	Running	Running	Running



Pictures
<https://jp.newsroom.ibm.com/2023-12-05-IBM-Debuts-Next-Generation-Quantum-Processor-IBM-Quantum-System-Two,-Extends-Roadmap-to-Advance-Era-of-Quantum-Utility>
<https://www.quantinuum.com/products-solutions/quantinuum-systems/system-model-h1>
https://www.riken.jp/medialibrary/riken/pr/news/2023/20231005_1_photo1.jpg

“A” RIKEN Superconductive Quantum Computer

- RIKEN Center for Quantum Computing
 - The successor of the *first* developed superconducting qubit
 - The cloud service started in Mar. 2023
 - 16 → 64 (now) → 144 qubits
→ 256 qubits (Fujitsu)

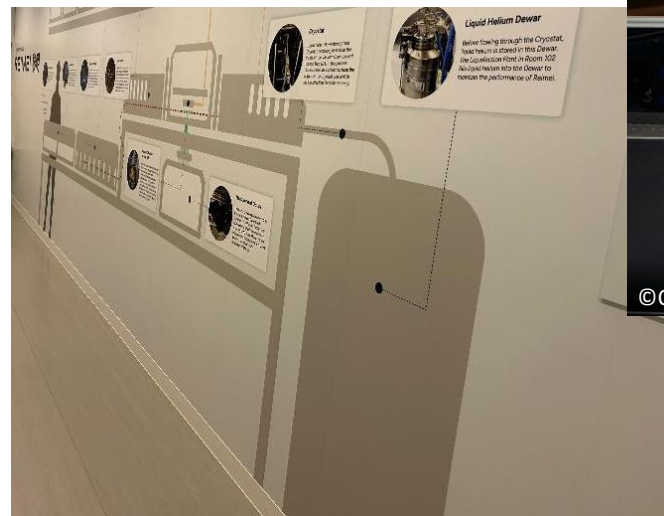
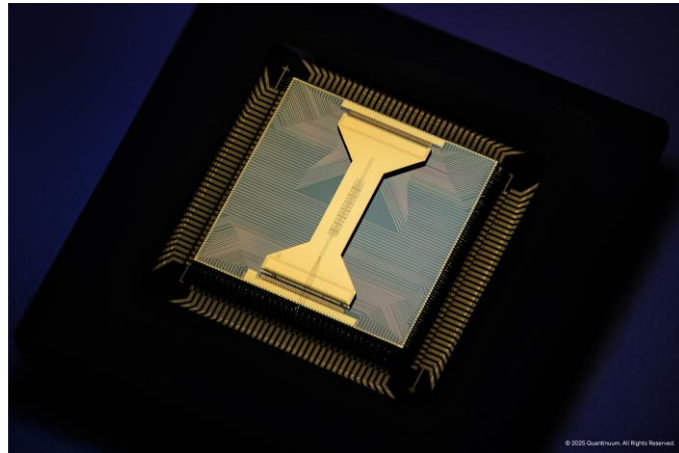


“Reimai” Quantinuum Trapped-Ion Quantum Computer

Quantinuum Trapped-Ion Quantum Computer H1 (20 qubits) was installed in Wako Campus, Japan and is operated since Feb, 2025



QUANTINUUM
REIMEI



IBM Superconductive Quantum Computer



- IBM Heron
 - 156-qubit

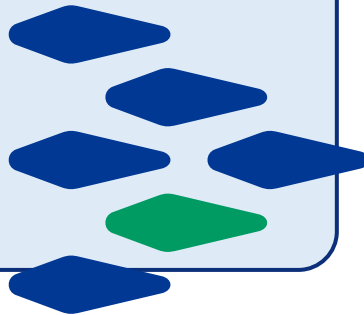
Quantum HPC related Projects in RIKEN R-CCS

TRIP3 (2024.04-

Expansion of Computationally Viable Region,
Transformative Research Innovation Platforms of
RIKEN Platforms,

A cross-disciplinary project to establish
connections among RIKEN's cutting-edge research
platforms such as *supercomputers*, *quantum
computing*, large synchrotron radiation facilities,
bioresource-projects.

R-CCS and Center for Quantum Computing (RQC)
are tasked with “the value creation by quantum-
HPC hybrid computing for Accelerating Research
DX”.

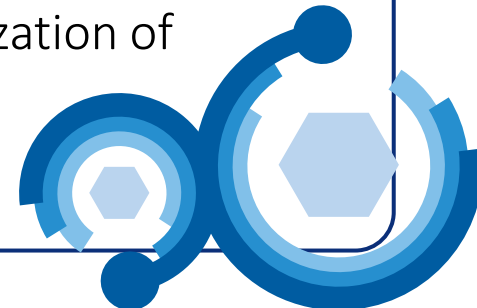


JHPC-Quantum (2024.11-

Research and Development of Quantum-
Supercomputers Hybrid Platform for Exploration of
Uncharted Computable Capabilities

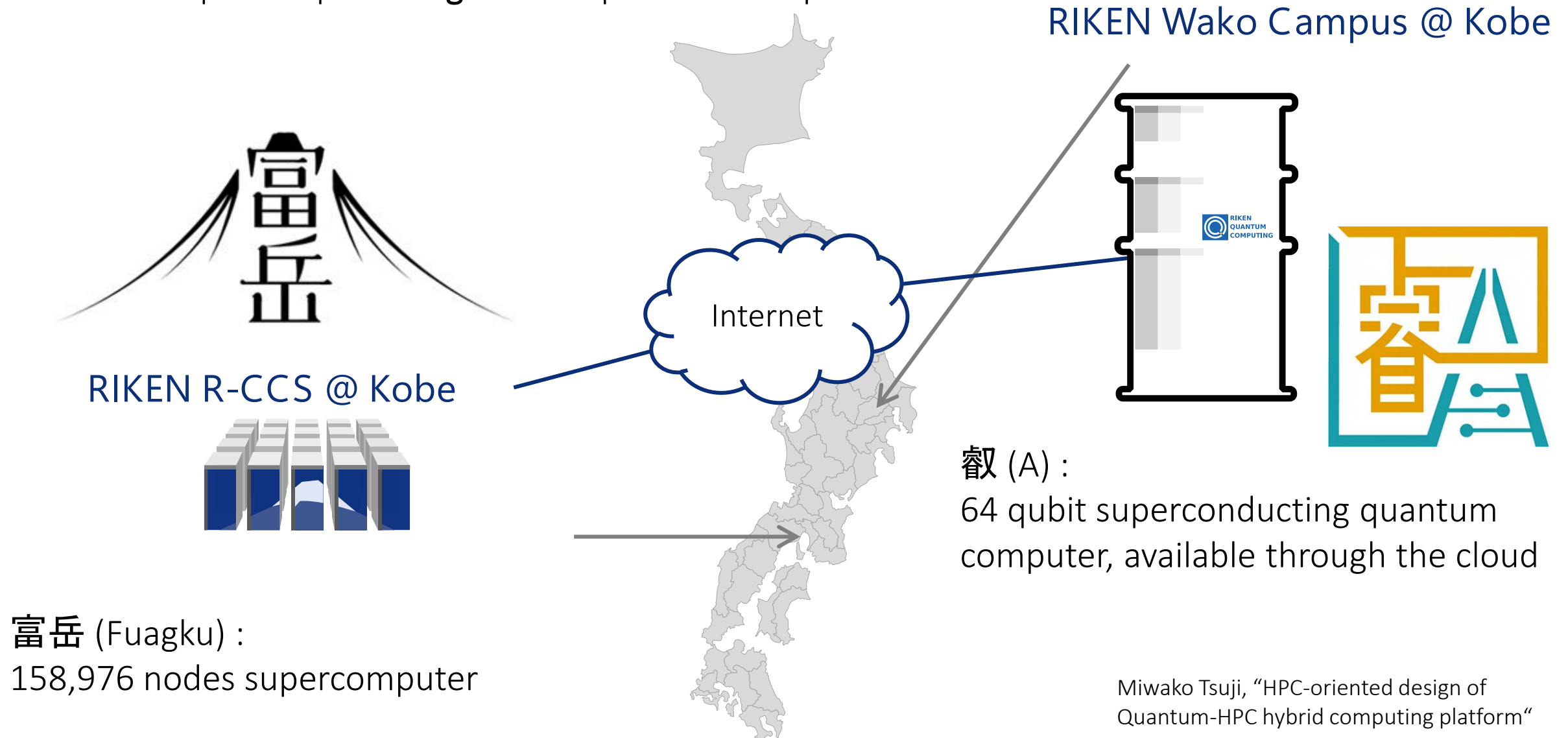
One of research projects in “Research and
Development Project of the Enhanced
Infrastructures for Post-5G Information and
Communication Systems” in NEDO, an agency
under the Ministry of Economy, Trade and Industry.
R-CCS, Softbank Corp., the university of Tokyo, and
Osaka university

A platform that connects supercomputers and
different types of quantum computers
to promote the early commercialization of
quantum computers.



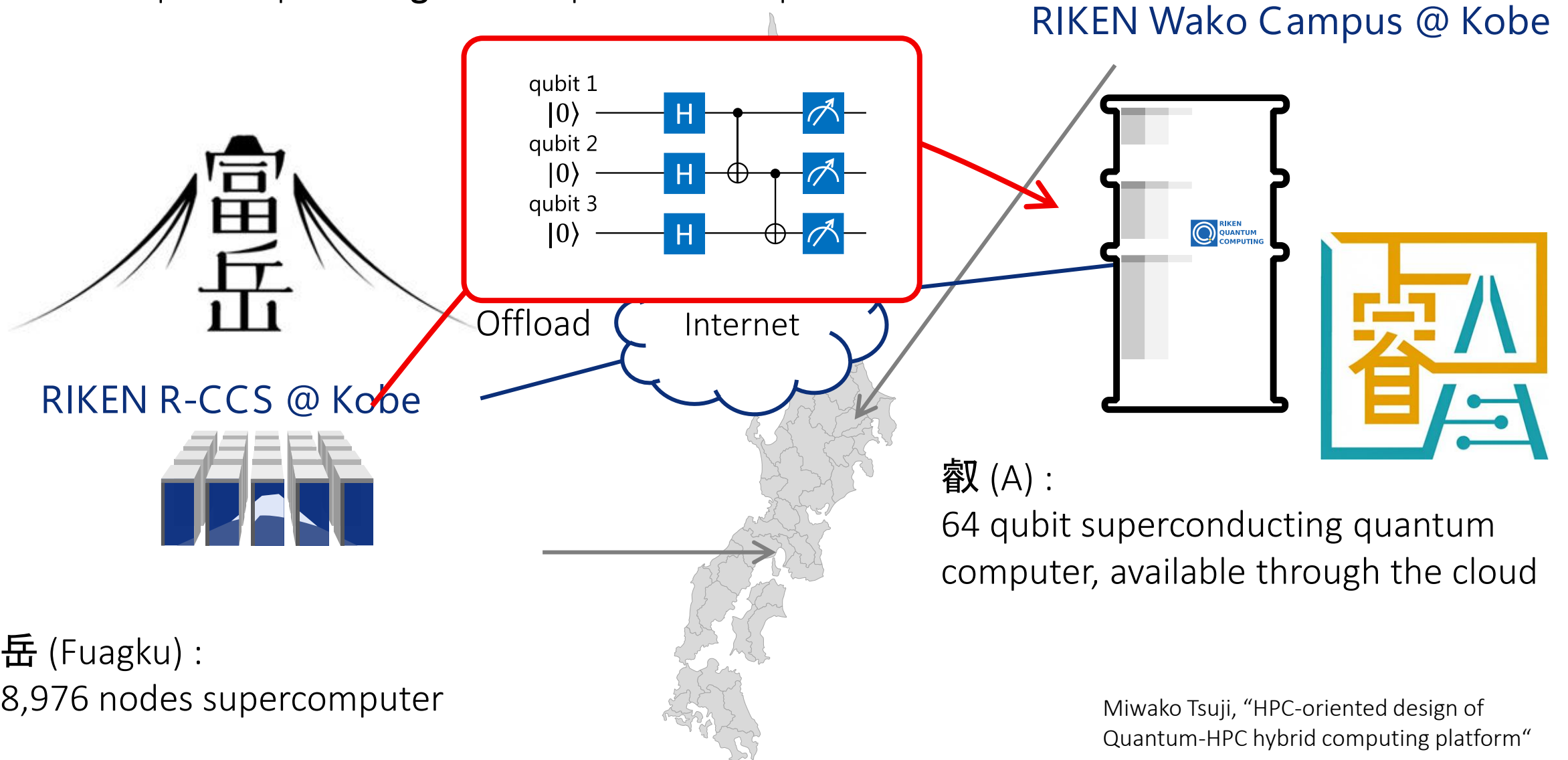
TRIP Proof of concept in 2023/2024 experiments and lessons learned

- Connect supercomputer **Fugaku** and quantum computer **A**



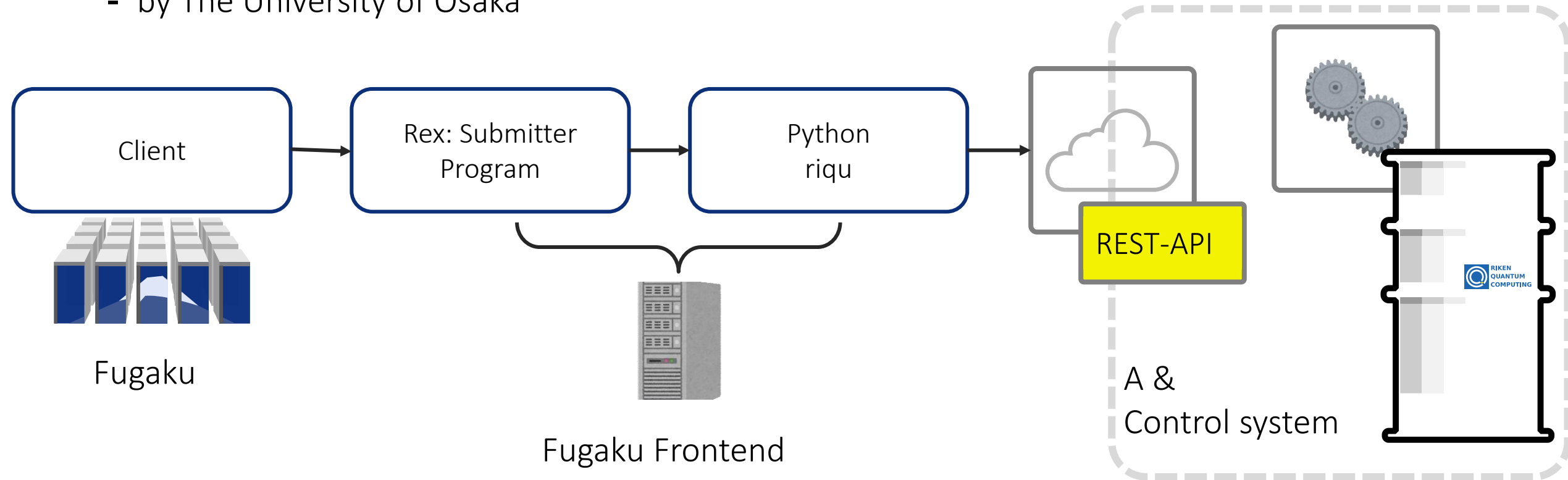
TRIP Proof of concept in 2023/2024 experiments and lessons learned

- Connect supercomputer **Fugaku** and quantum computer **A**



1st Experiment: RPC though a Python based library

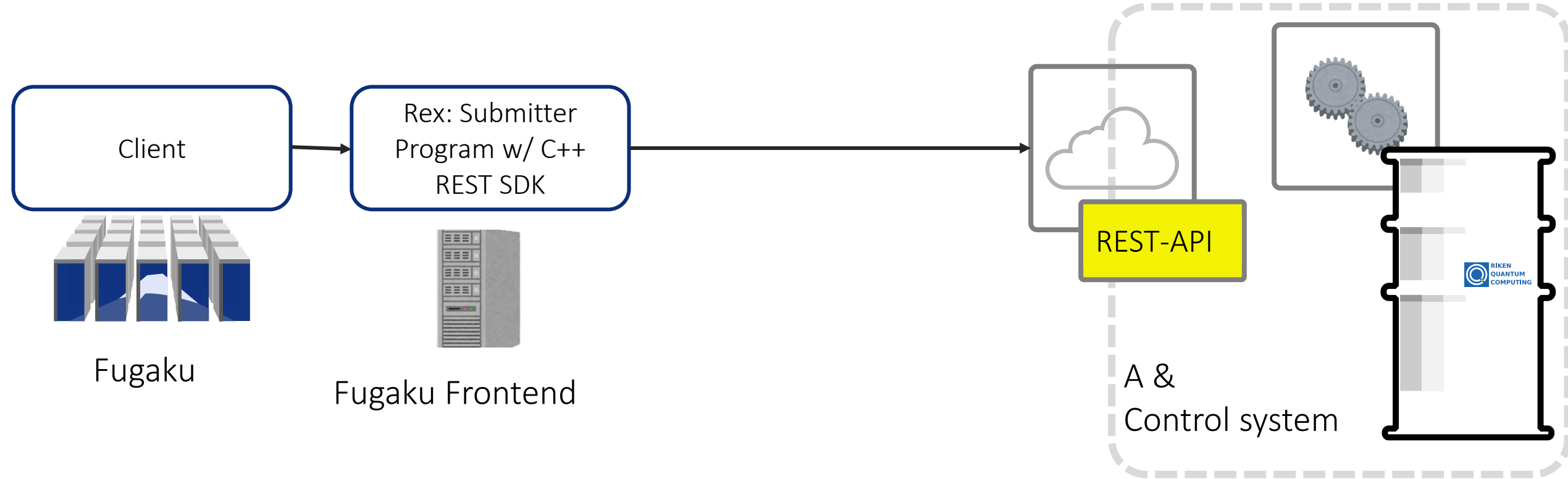
- A client program calls a remote submitter program
- The submitter program calls a Python library designed to use “A” through REST-API
- QURI Parts riqu: **R**EST Interface for **Q**uantum **C**omputing
 - by The University of Osaka



Note: 4 of 64 qubit are available for us

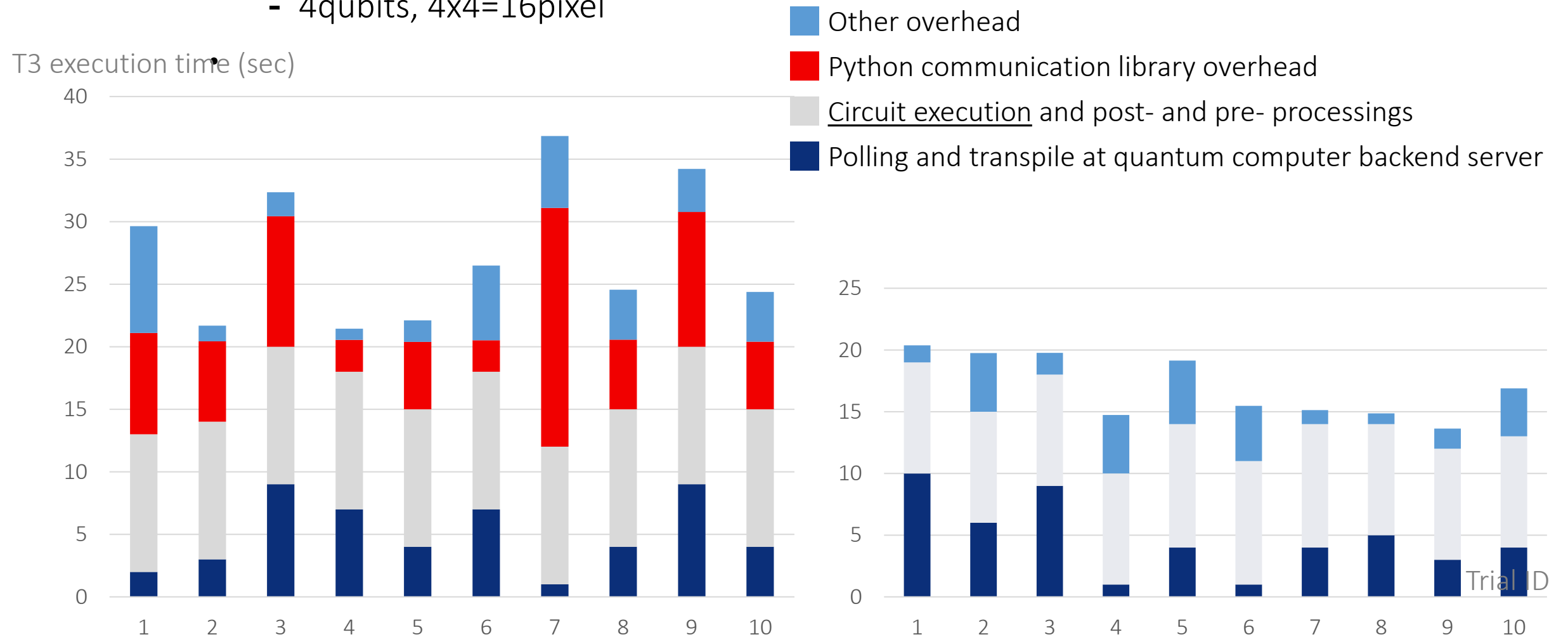
2nd Experiment: RPC though C++ based library

- The C++ REST SDK
 - <https://github.com/microsoft/cpprestsdk>



Result

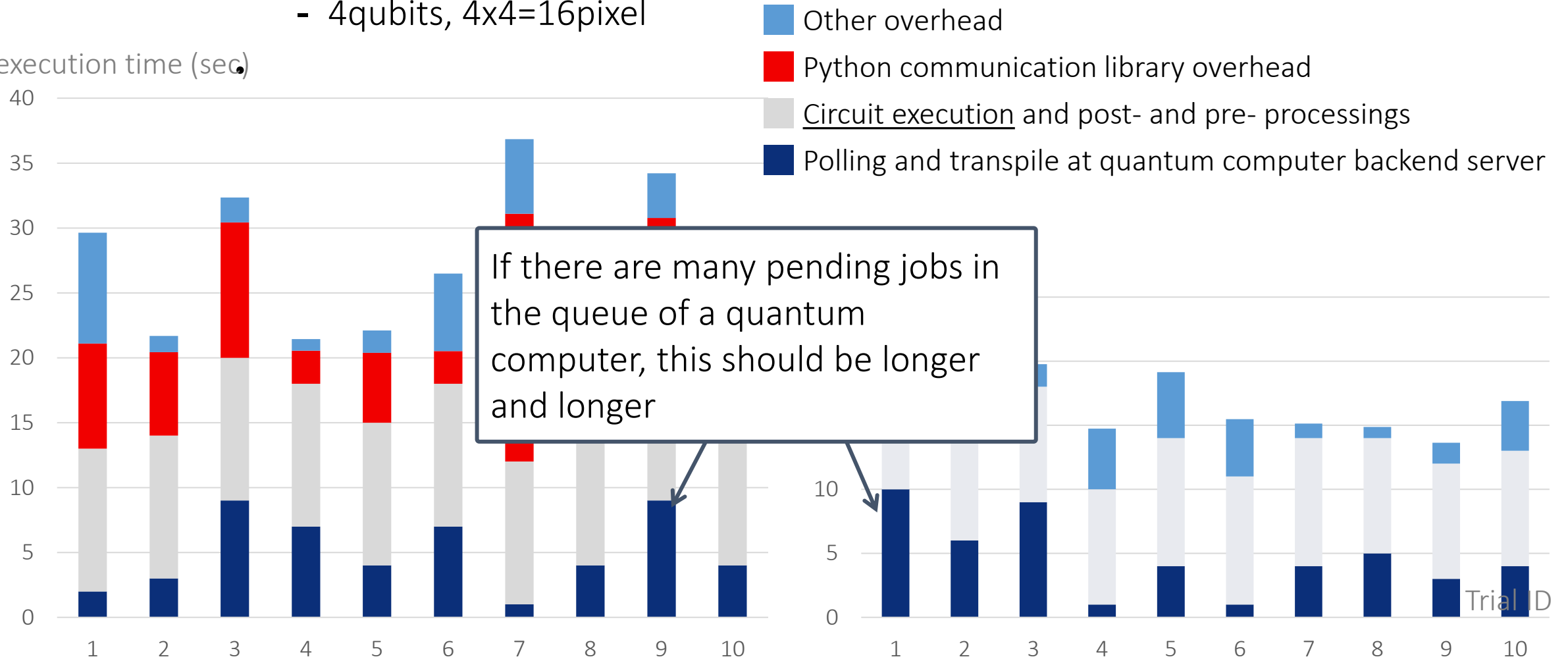
- Quantum Circuit Encoding Workflow
- A simple step-by-step workflow
- Automatic quantum circuit encoding (AQCE) of a given arbitrary quantum state
 - Generate a circuit to make a gray scale image:
 - 4qubits, 4x4=16pixel



Result

- Quantum Circuit Encoding Workflow
- A simple step-by-step workflow
- Automatic quantum circuit encoding (AQCE) of a given arbitrary quantum state
 - Generate a circuit to make a gray scale image:
 - 4qubits, 4x4=16pixel

T3 execution time (sec)



TRIP Proof of concept in 2023/2024 experiments and lessons learned

- We found our programming model cannot be used in our supercomputer system (esp “Fugaku”)
- Coordinated scheduling with quantum computer and supercomputer is required.
- Execution of HPC programs running multiple nodes should not be idle as possible as it can.
- Major programming environments of quantum computer program development are Python-based frameworks.
 - Python is difficult to be combined with HPC programs such as MPI, multithreaded programs in C/C++ and Fortran.
 - Recently, some quantum computing programming frameworks in C/C++ are proposed

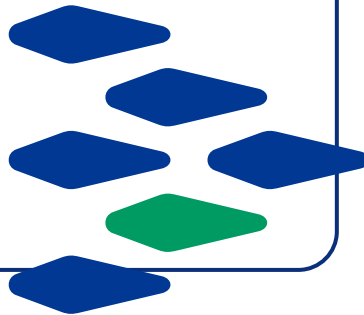
Quantum HPC related Projects in RIKEN R-CCS

TRIP3 (2024.04-

Expansion of Computationally Viable Region,
Transformative Research Innovation Platforms of
RIKEN Platforms,

A cross-disciplinary project to establish
connections among RIKEN's cutting-edge research
platforms such as *supercomputers*, *quantum
computing*, large synchrotron radiation facilities,
bioresource-projects.

R-CCS and Center for Quantum Computing (RQC)
are tasked with “the value creation by quantum-
HPC hybrid computing for Accelerating Research
DX”.

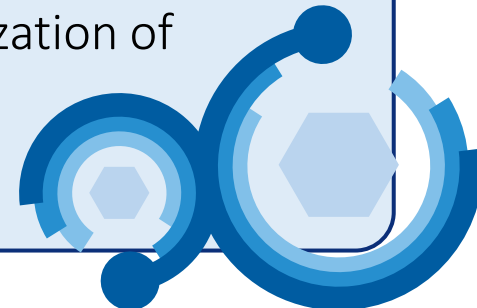


JHPC-Quantum (2024.11-

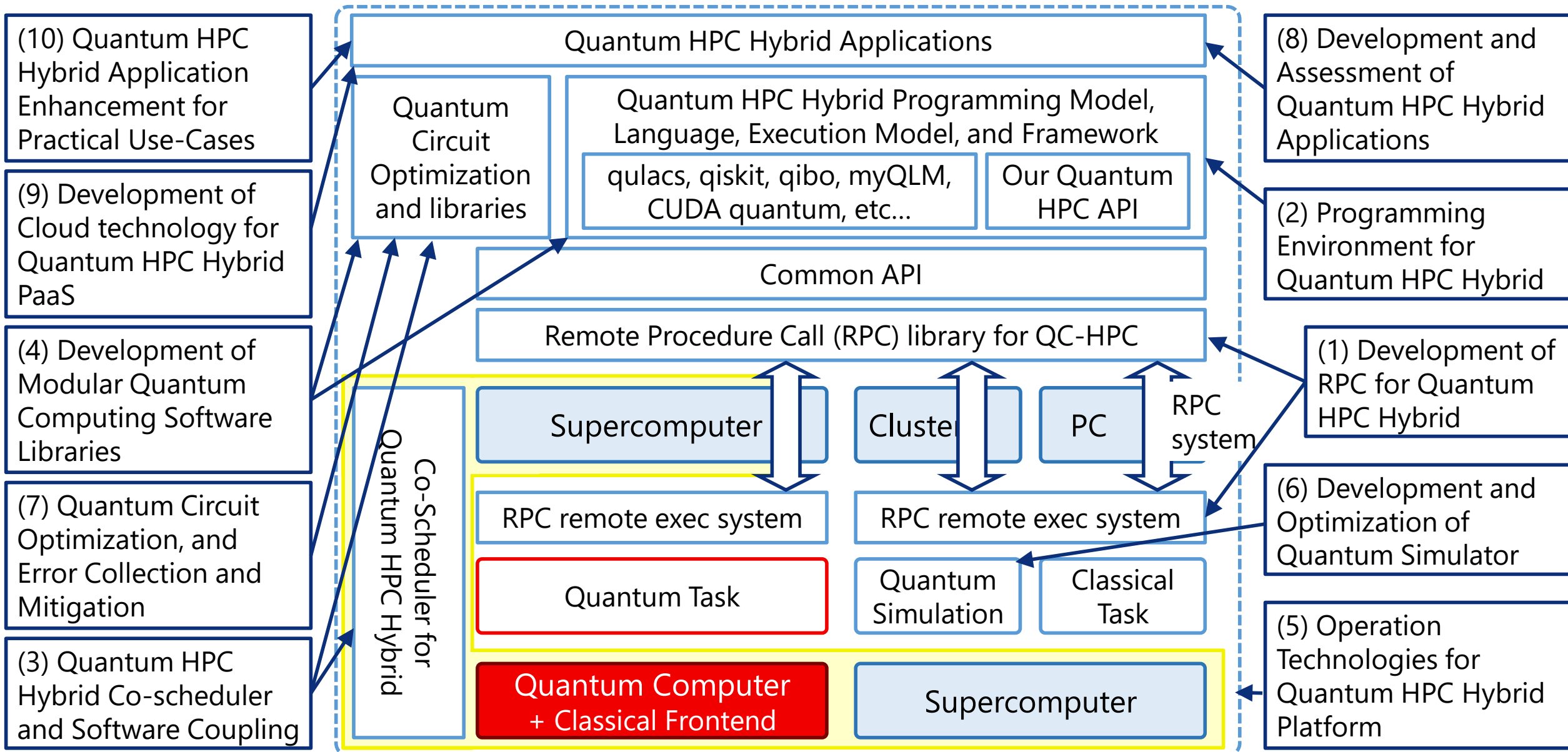
Research and Development of Quantum-
Supercomputers Hybrid Platform for Exploration of
Uncharted Computable Capabilities

One of research projects in “Research and
Development Project of the Enhanced
Infrastructures for Post-5G Information and
Communication Systems” in NEDO, an agency
under the Ministry of Economy, Trade and Industry.
R-CCS, Softbank Corp., the university of Tokyo, and
Osaka university

A platform that connects supercomputers and
different types of quantum computers
to promote the early commercialization of
quantum computers.

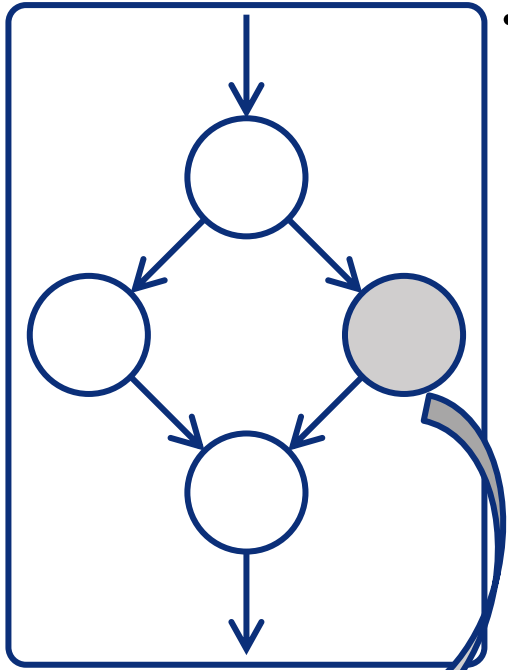


Research Items in JHPC-Quantum Project



Quantum HPC Middleware: Two Level Programming Model

- Workflow & Task based parallelism



- Workflow

- Tasks (different kernels) are managed by a workflow engine
- The tasks are “**jobs**” in supercomputers and/or quantum computer
- The workflow engine submit each of jobs in a certain order based on the descriptions about dependencies between jobs

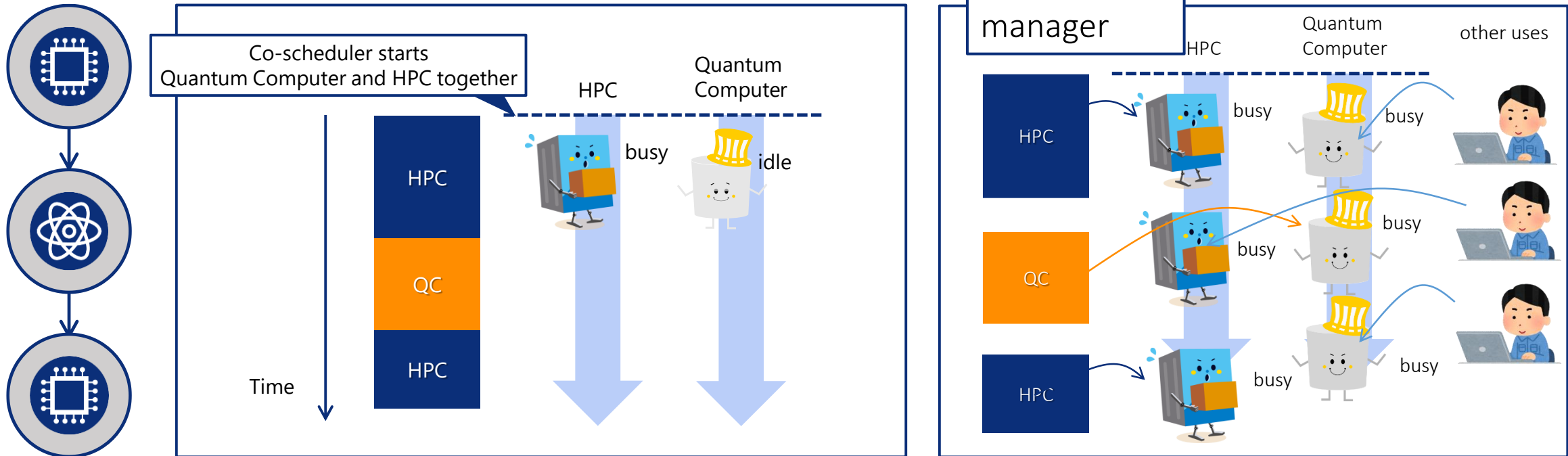
Simple HPC Job 	Simple QC Job 	HPC-QC Job 
running a kernel only on HPC	offloads a quantum circuit to quantum computer	includes both of HPC and quantum kernels

- Task based Parallelism in HPC-QC Jobs

- offloads quantum kernel(s) to a quantum computer via remote procedure call (RPC)
- A single HPC-QC job may offload multiple kernels
- RPC is an asynchronous mechanism; Other classical kernels will be executed on CPU simultaneously

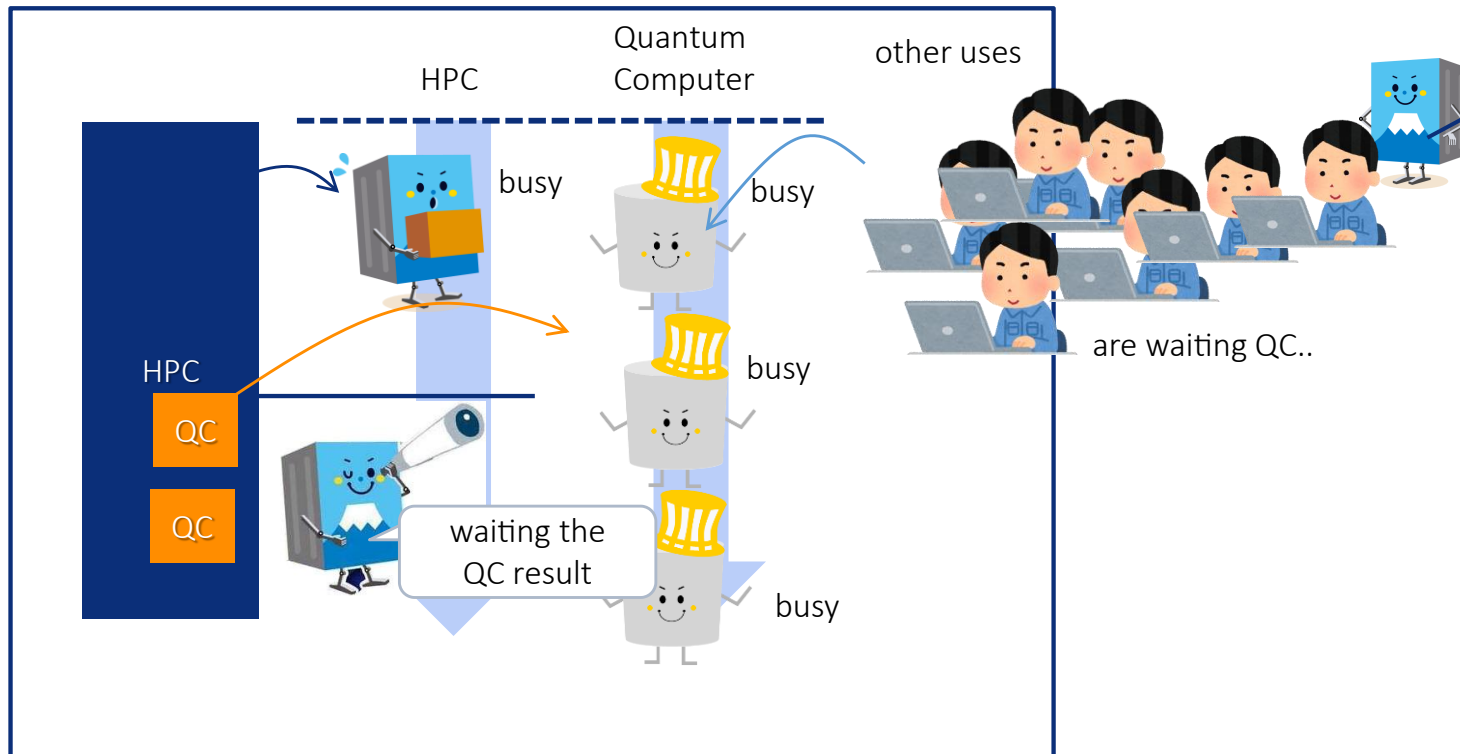
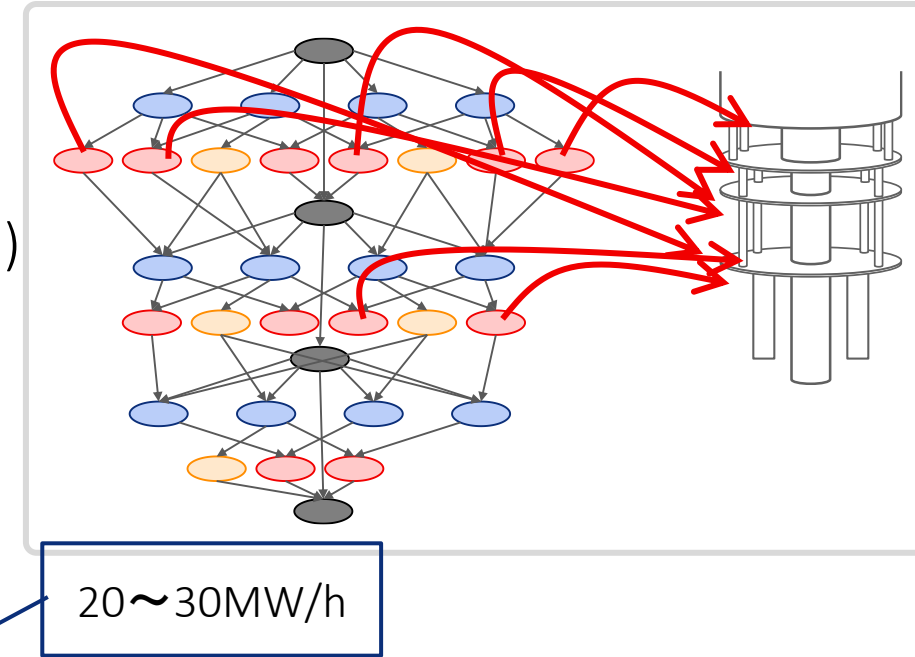
Why do we need the 1st Layer, Job-Scheduler Based Workflow?

- Avoid the waiting times in quantum computer and supercomputer
 - Uses separate quantum kernels and HPC kernels in their applications
 - Define the order of their execution
- Job-schedulers in both computers may arrange jobs to maximize the throughput in their systems.
- A job may have to wait the result from another job, but the waiting time does not consume any computational resource



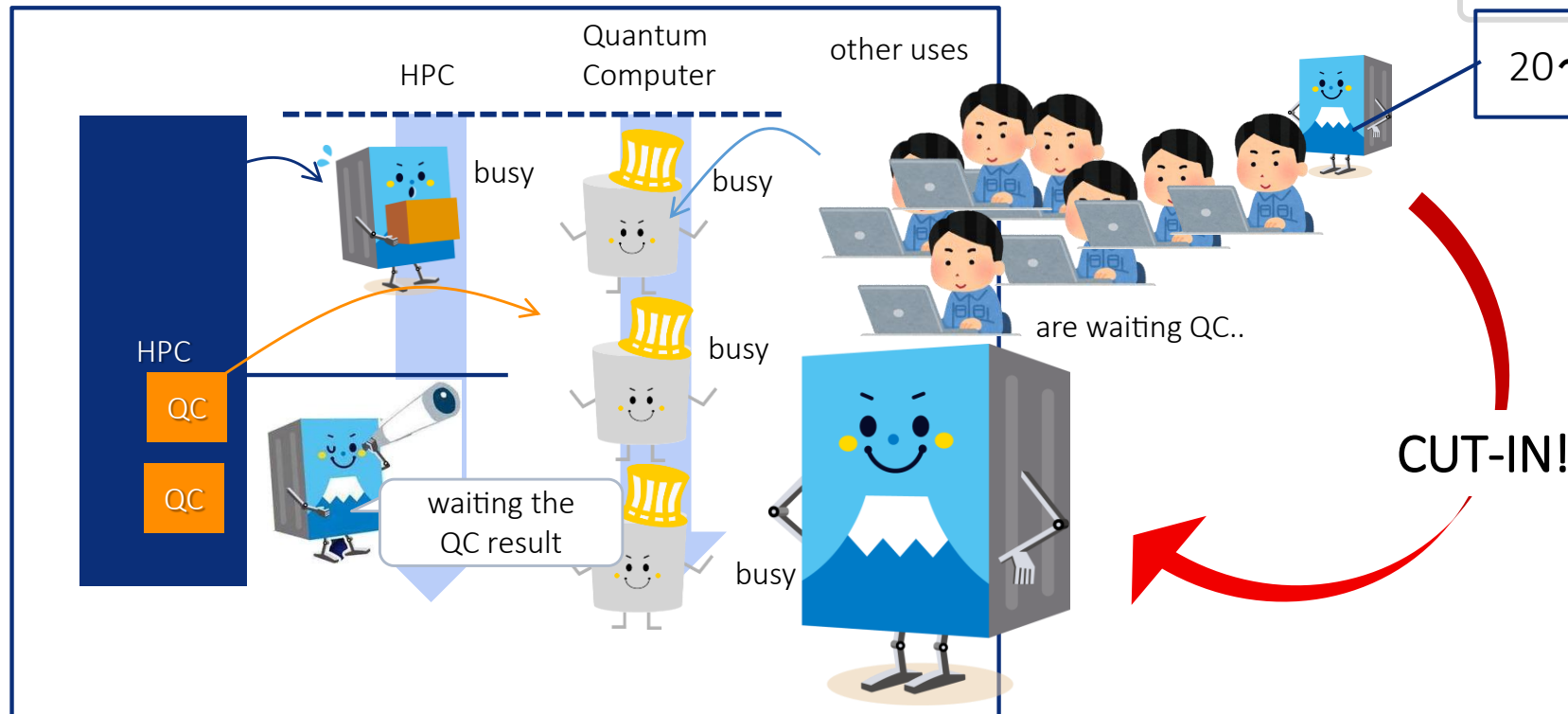
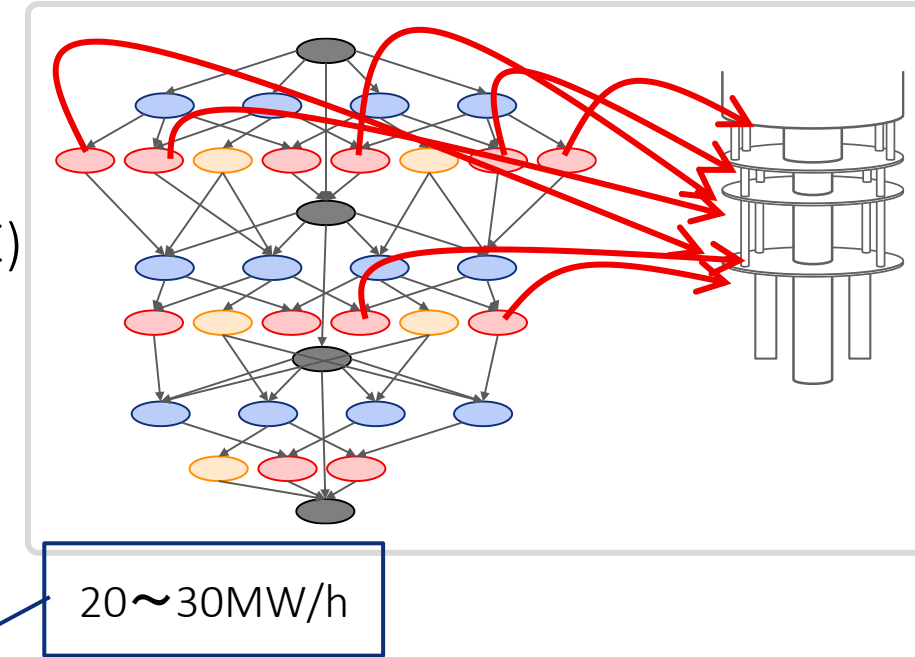
Why do we need 2nd Layer, task-based programming

- Tightly coupled quantum and HPC kernels in a single job
 - Difficult to separate quantum and HPC kernels completely
- Multiple quantum kernel calls from HPC in a short time (ex. VQE)
 - Difficult to wait each of the quantum circuit executions
- (Still) We must avoid the waiting time in HPC, especially for large scale HPC jobs



Why do we need 2nd Layer, task-based programming

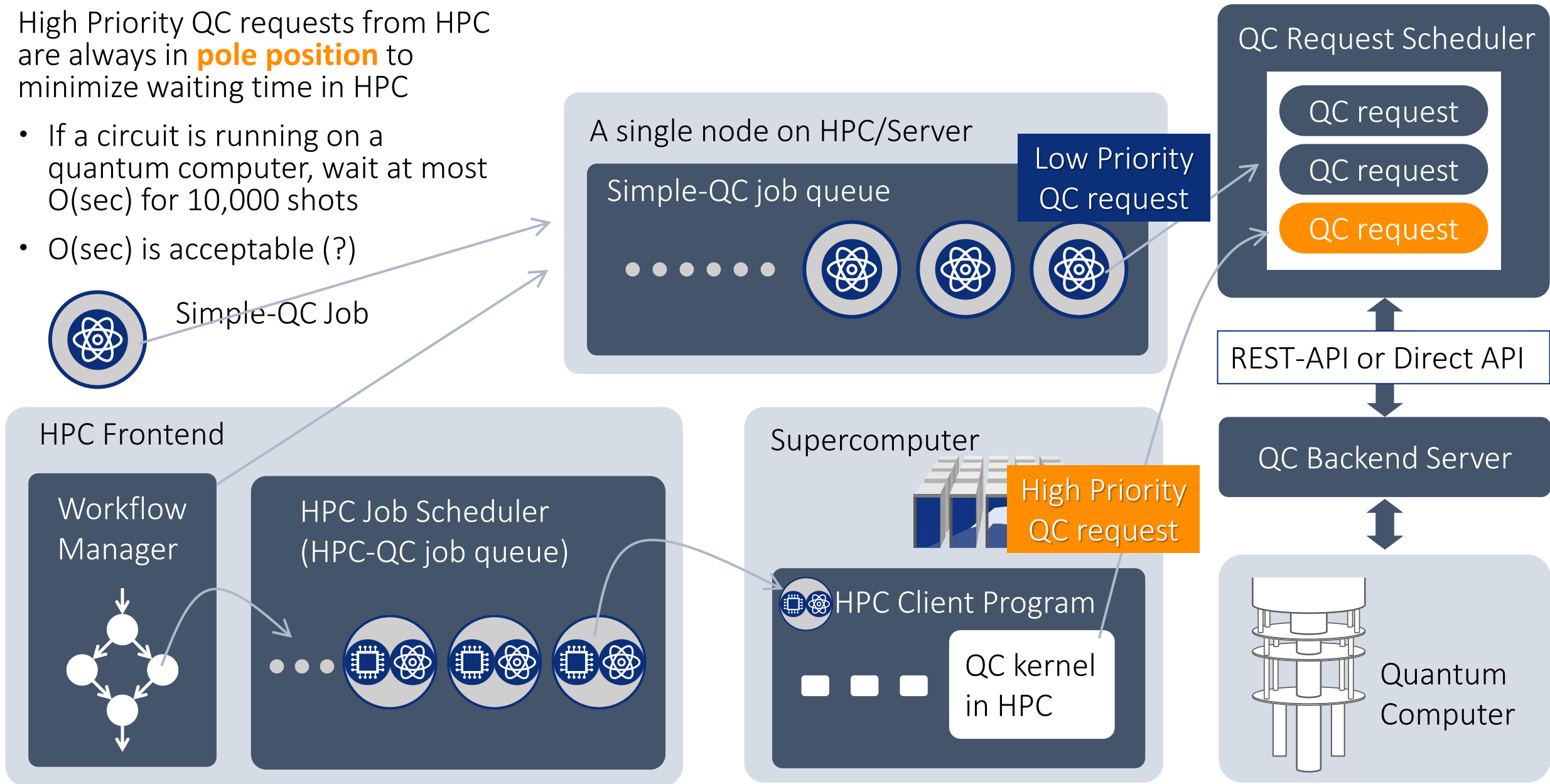
- Tightly coupled quantum and HPC kernels in a single job
 - Difficult to separate quantum and HPC kernels completely
- Multiple quantum kernel calls from HPC in a short time (ex. VQE)
 - Difficult to wait each of the quantum circuit executions
- (Still) We must avoid the waiting time in HPC, especially for large scale HPC jobs



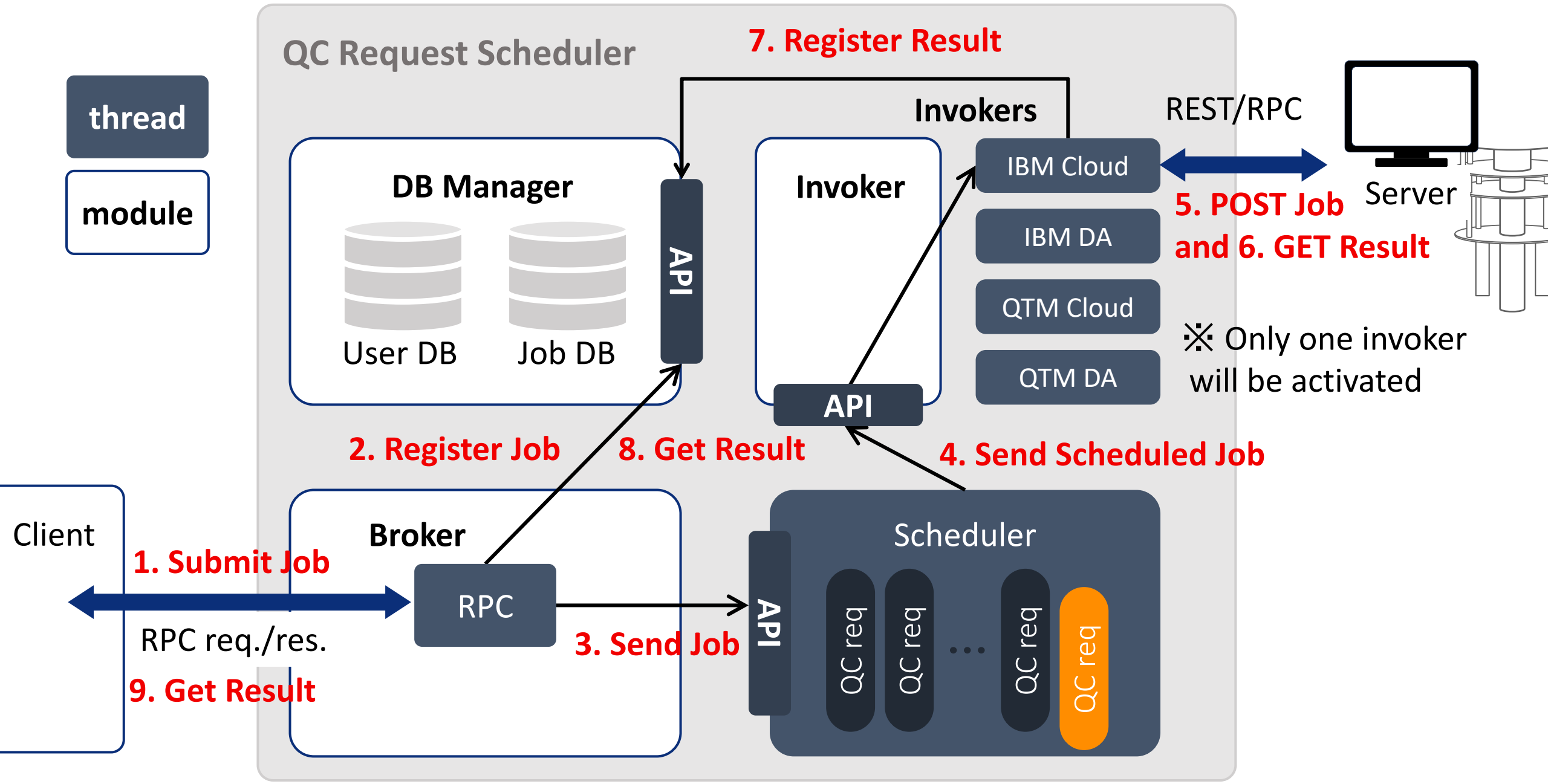
Execution Environment

High Priority QC requests from HPC are always in **pole position** to minimize waiting time in HPC

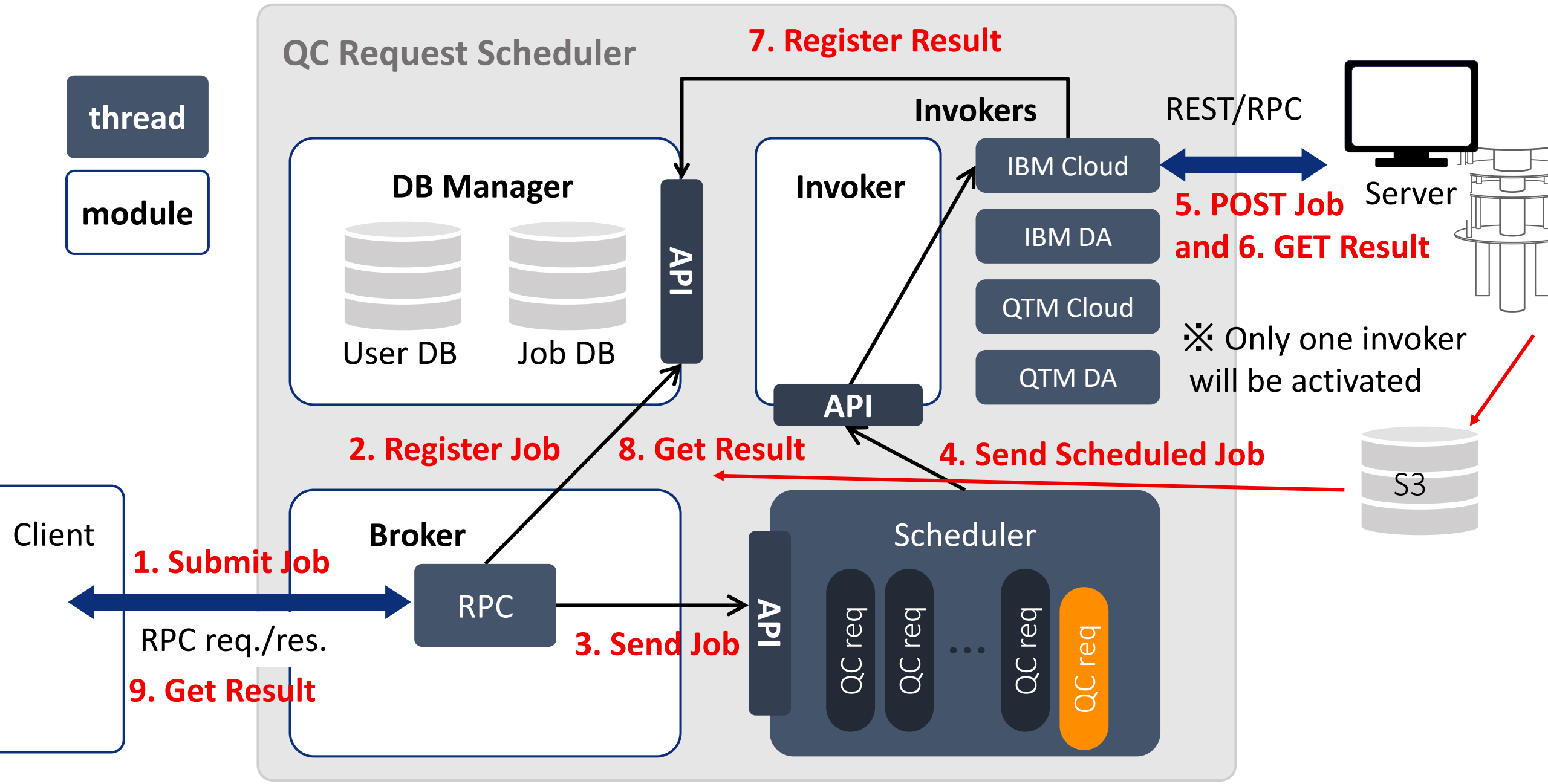
- If a circuit is running on a quantum computer, wait at most $O(\text{sec})$ for 10,000 shots
- $O(\text{sec})$ is acceptable (?)



SQC (Supercomputer Quantum computer Computation) Scheduler

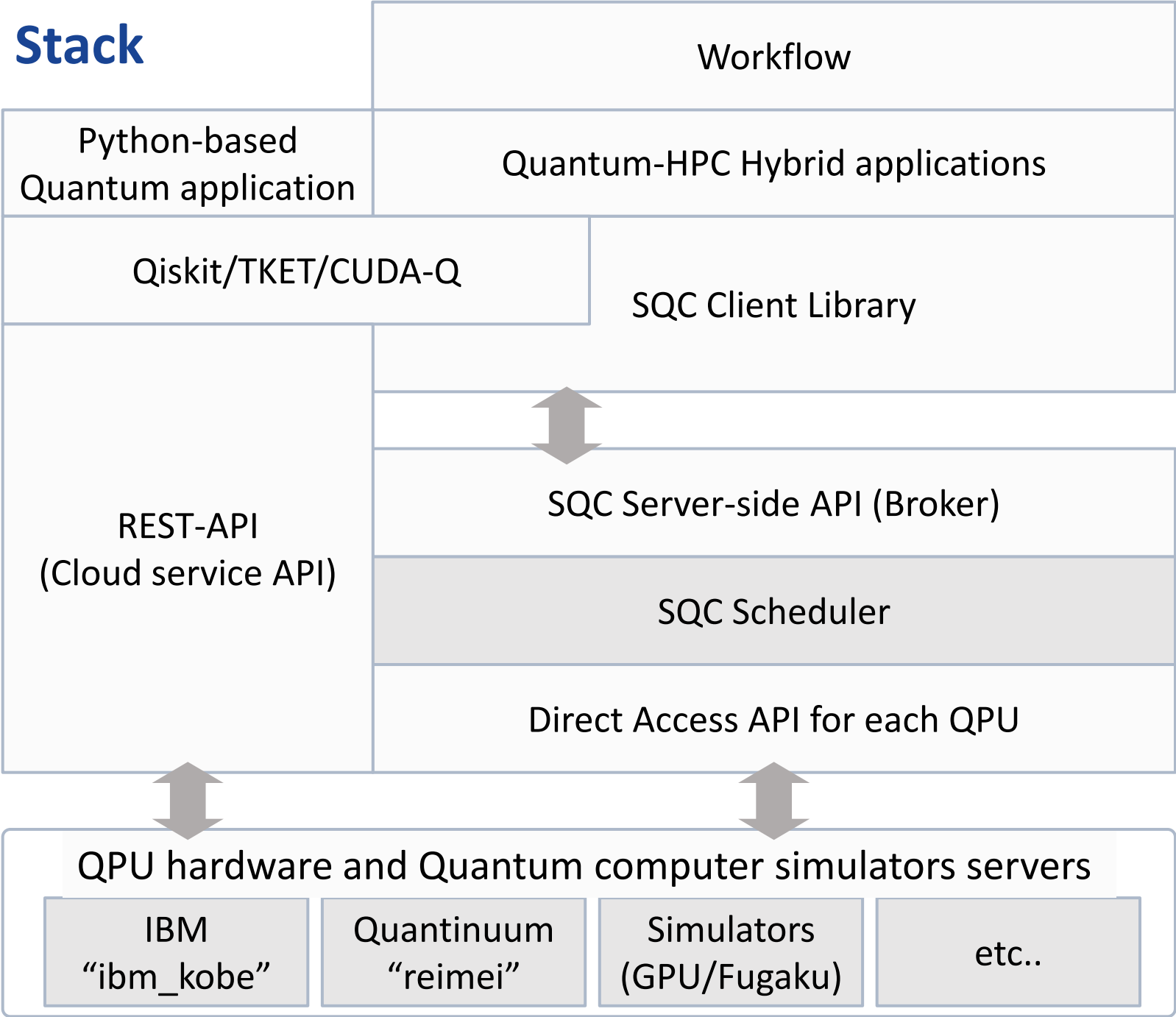


SQC Scheduler for IBM_Kobe



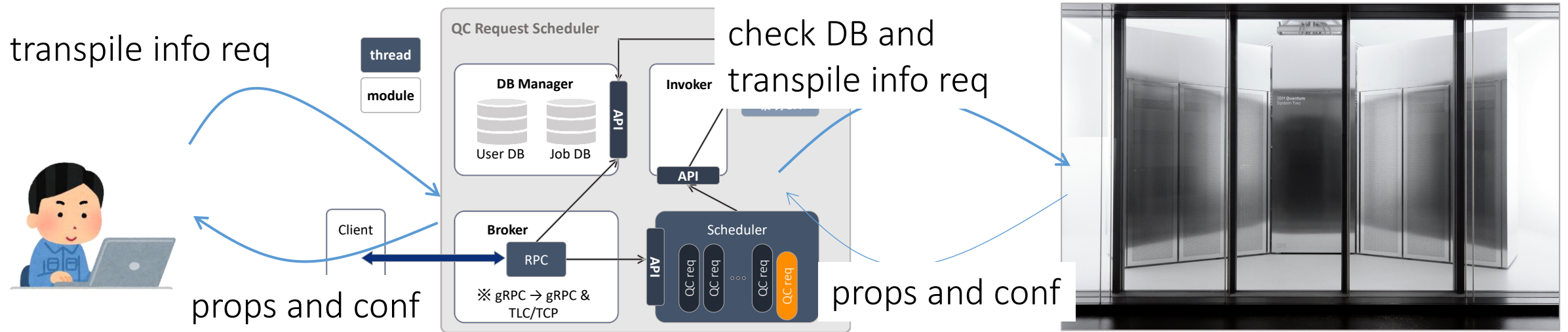
JHPC-Quantum Software Stack

- Two ways to access QC
 - Our SQC APIs and RPC. The RPC uses direct access APIs provided by QPU venders
 - REST-APIs through vender clouds
- Workflow tools
 - Tierkreis (Quantinuum), Prefect, Xcrypt
- QC software libraries
 - JHPC Quantum QC software lib
 - Qunasys QURI SDK
 - Error mitigation
 - Error suppression



Additional API for ibm_kobe, Transpile

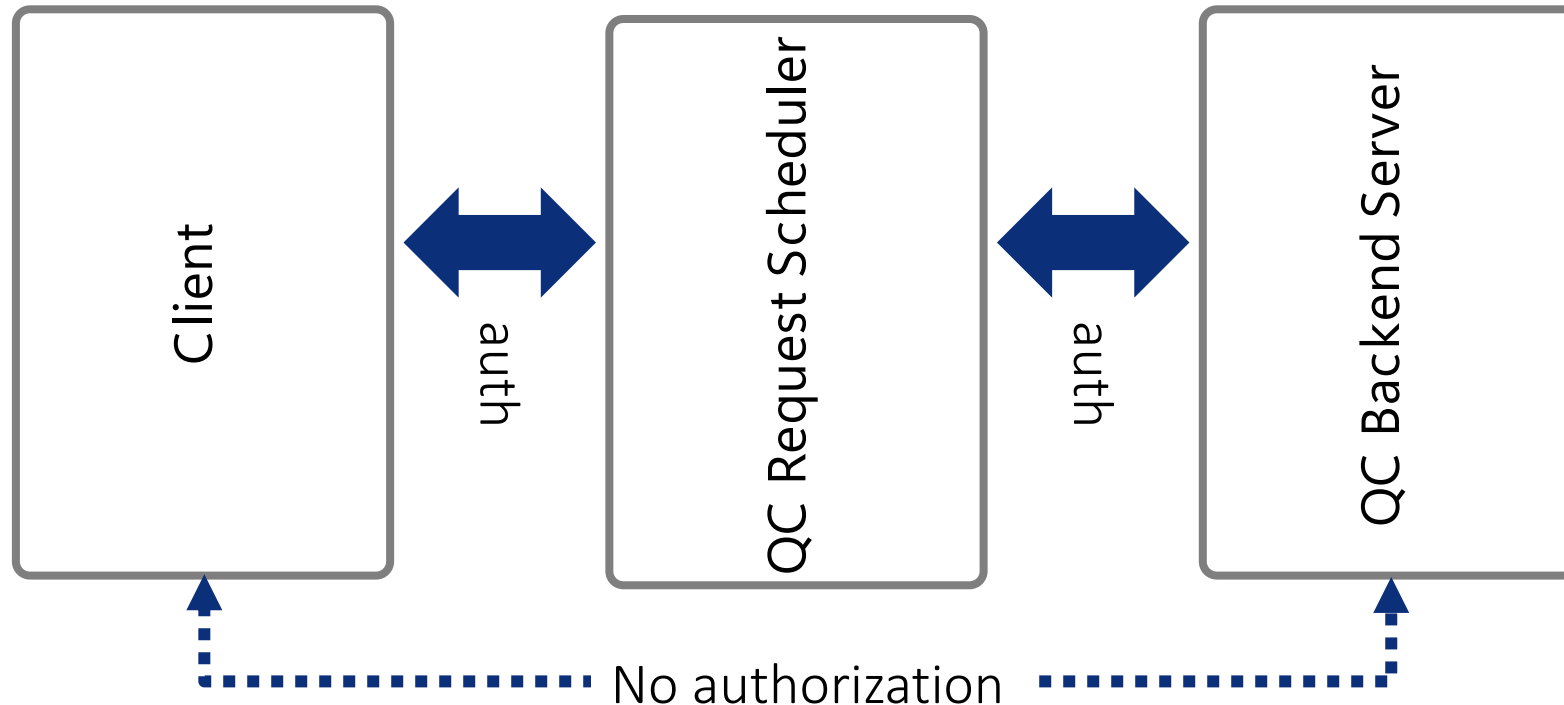
- `sqcIbmdTranspileInfo(circ, backend);`
 - User sends a request to SQC Request Scheduler to get backend property and configuration
 - results will be stored at `circ->backend_config_json` and `circ->backend_props_json`



- `sqcTranspile(circ, backend, topt);`
 - SQC Library transpiles a circuit by calling Qiskit python library internally
- Users can provide the above QPU data to their own transpilers and transpile circuits

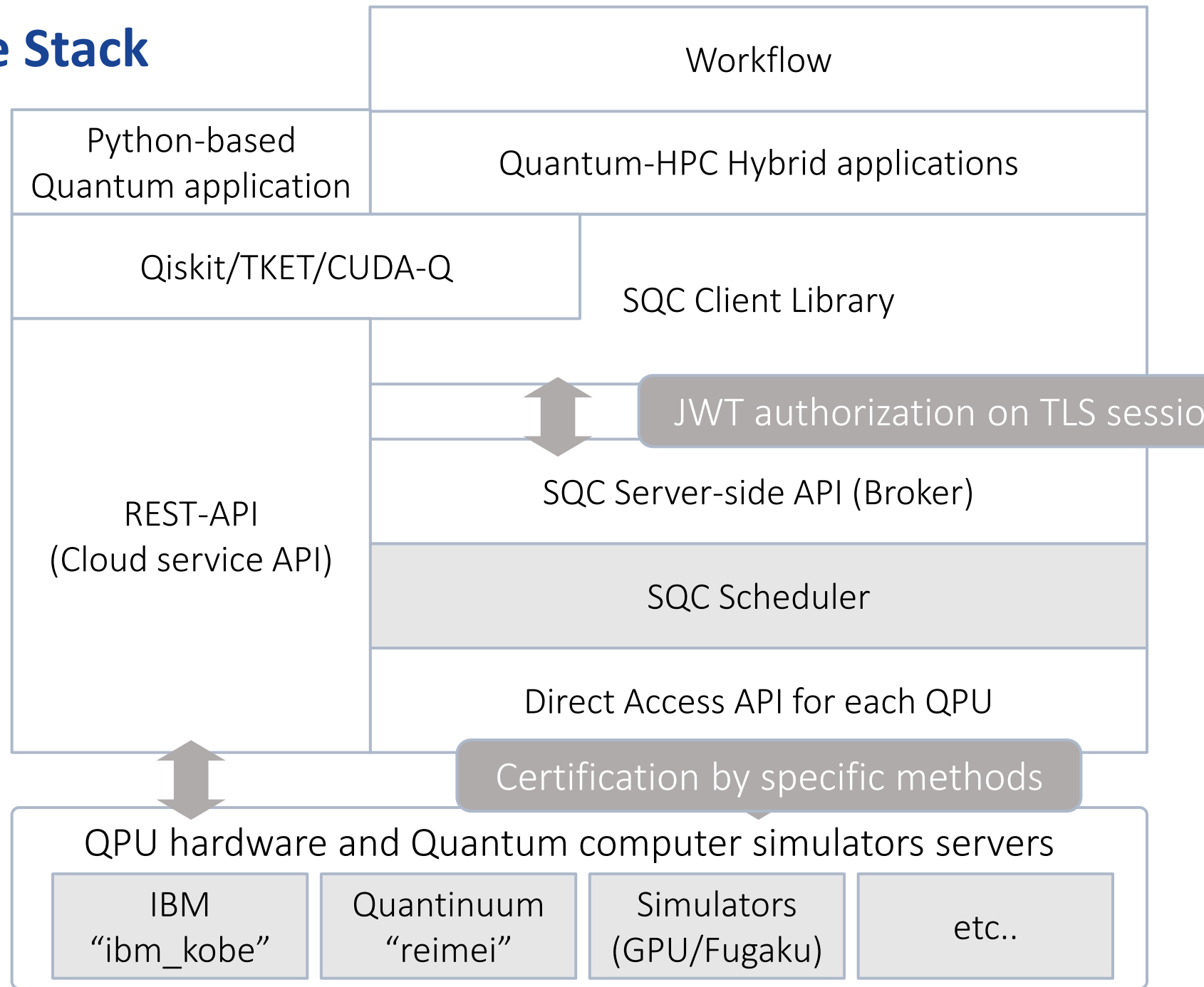
Authorization

- Client $\leftrightarrow$ SQC Scheduler
 - JSON Web Token (JWT) + TLS client certification
 - At the beginning of the session, JWT authentication is carried out over a TLS server-authenticated channel to guarantee communication confidentiality and integrity.
- SQC Scheduler $\leftrightarrow$ Backend Server for quantum computers
 - Authorization method specified by each quantum vendor
 - No authorization for end users



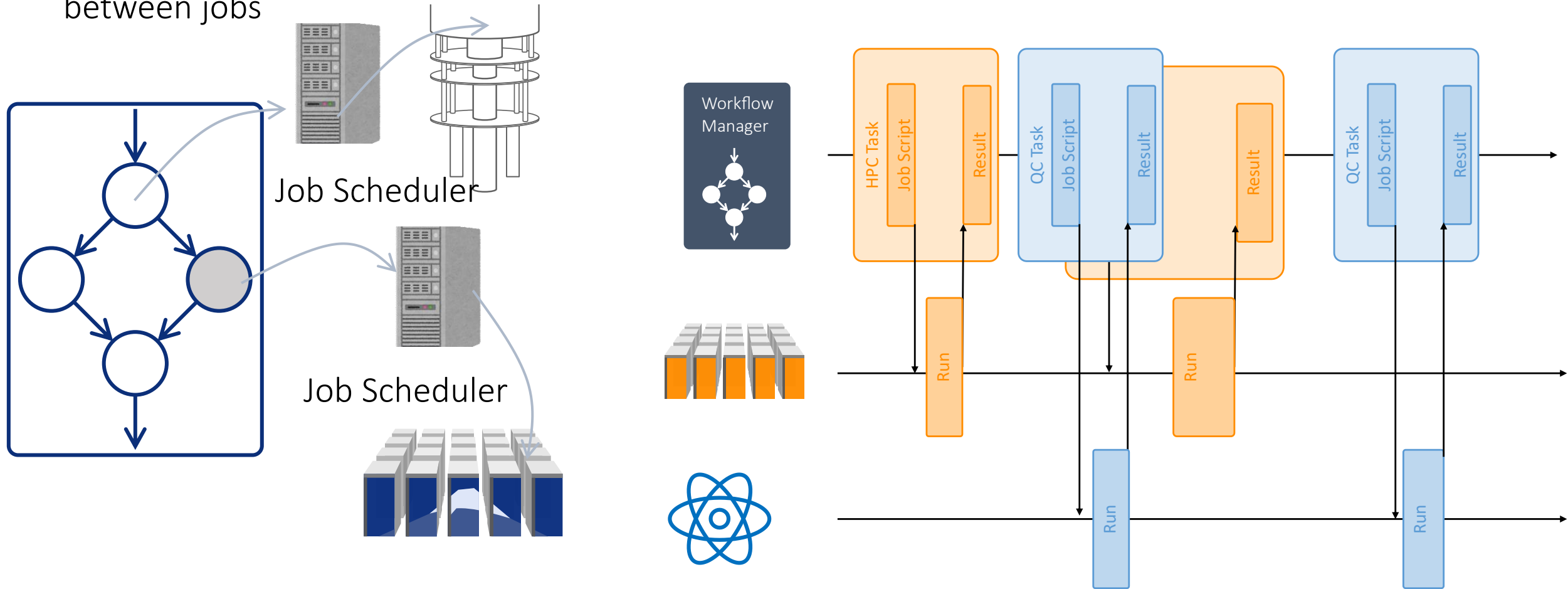
JHPC-Quantum Software Stack

- Two ways to access QC
 - Our SQC APIs and RPC. The RPC uses direct access APIs provided by QPU vendors
 - REST-APIs through vender clouds
- Workflow tools
 - Tierkreis (Quantinuum), Prefect, Xcrypt
- QC software libraries
 - JHPC Quantum QC software lib
 - Qunasys QURI SDK
 - Error mitigation
 - Error suppression



Quantum-HPC Workflow: Coarse Grained Task-based Programming

- Tasks are large, typically, whole programs, jobs in supercomputers or quantum computers
- A workflow manager will take care about the order of job submissions based on the dependency between jobs



Fine Grained Task based Programming: Example of C/C++ Program on SQC library

```
#include<stdio.h>
...
#include"sqc_api.h"

int main(int argc, char **argv)
{
    const int nqubits = 10;
    sqcInitOptions      *iopt;
    iopt = sqcMallocInitOptions();
    sqcInitialize(iopt);

    sqcQC* circ;
    circ = sqcQuantumCircuit(nqubits);
    sqcHGate (circ, 1);
    sqcCXGate(circ, 0, 1);
    for(int i=0; i<nqubits; i++)
        sqcMeasure(circ, i, i, NULL);

    sqcBackend backend = SQC_RPC_SCHED_QC_TYPE_QTM_SIM_GRP;
    sqcOut result;

    circ->qasm = (char *)malloc((circ->ngates)*128);
    int len = sqcConvQASMtoMemory(circ, backend,
                                   circ->qasm, len);

    if(len <= 0) { something;}
```

```
sqcTranspile(circ, backend, &ropt);

sqcRunOptions      ropt;
sqcInitializeRunOpt(&ropt);
ropt.nshots = 30;
ropt.qubits = nqubits;

int ret = sqcQCRun(circ, backend, ropt, &result);

sqcPrintQCResult(stdout, &result, ropt.outFormat);
sqcFreeOut(&result, ropt.outFormat);

sqcDestroyQuantumCircuit(circ);
sqcFinalize(iopt);
sqcFreeInitOptions(iopt);
....*
```

Example of Qiskit

```
from qiskit.transpiler.preset_passmanagers import
generate_preset_pass_manager
from qiskit_ibm_runtime.utils.backend_converter
import convert_to_target
from qiskit_ibm_runtime.models import
BackendProperties, BackendConfiguration
from qiskit.circuit import QuantumCircuit
from qiskit_sqc_runtime import SQCBackend,
SQCSamplerV2
from qiskit import qasm3
```

```
import json
import codecs
```

```
# Construct circuit
qc = QuantumCircuit(3, 3)
qc.h(0)
qc.cx(0, 1)
qc.cx(0, 2)
qc.measure(0,0)
qc.measure(1,1)
qc.measure(2,2)
```

```
# Get backend
backend = SQCBackend("ibm-dacc")
```

```
# Run quantum circuit
sampler = SQCSamplerV2(backend)
transpile_info = backend.transpile_info()
```

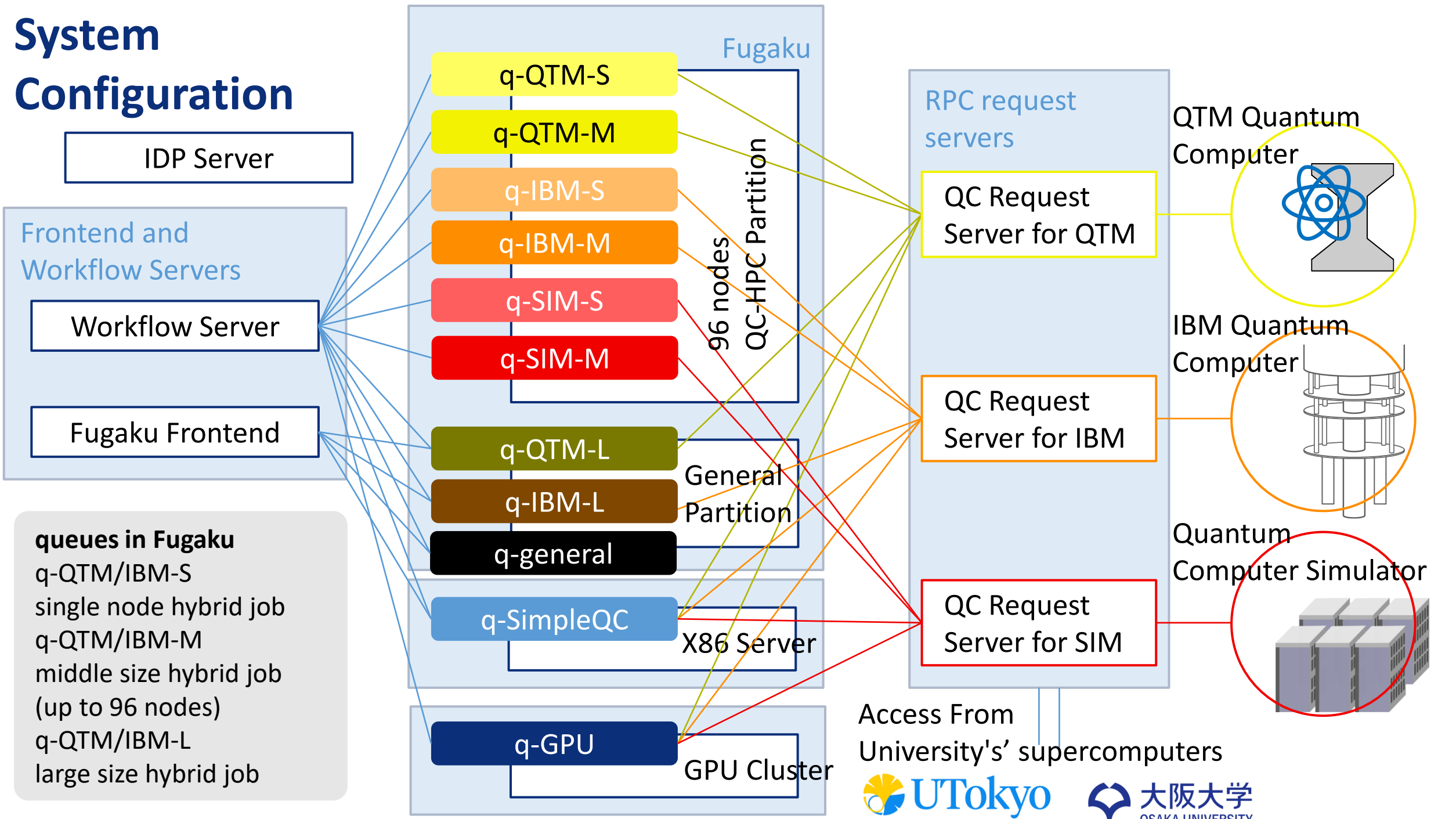
```
prop_json = json.loads(transpile_info[0])
backend_prop =
BackendProperties.from_dict(prop_json)
conf_json = json.loads(transpile_info[1])
backend_conf =
BackendConfiguration.from_dict(conf_json)
```

```
target = convert_to_target(backend_conf,
backend_prop)
pm = generate_preset_pass_manager(
    optimization_level=1,
    target=target,
)
isa_circuit = pm.run(qc)
```

```
job = sampler.run(isa_circuit, shots=10)
```

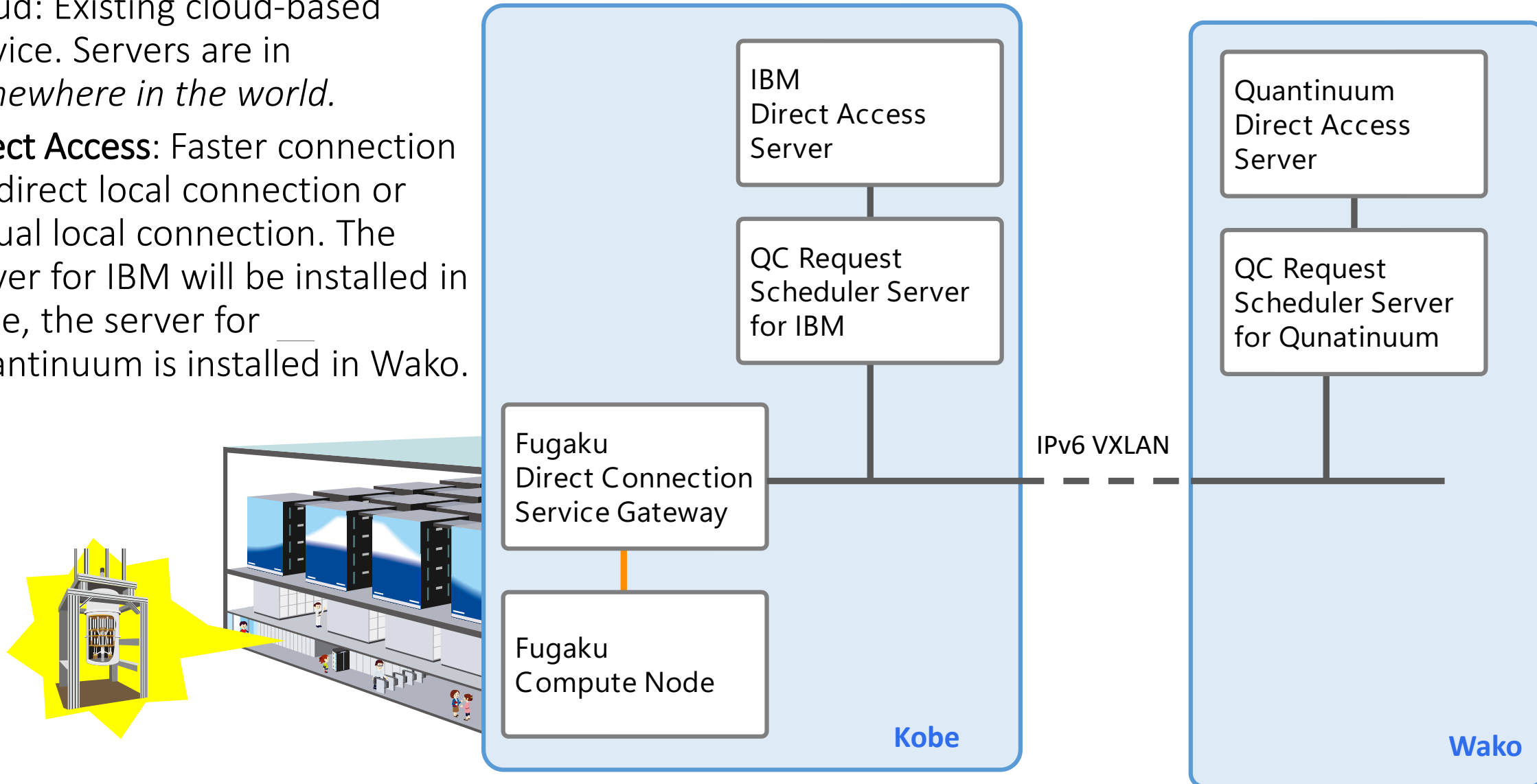
```
# Show result
result = job.result()
print(result)
print(job.status())
```

System Configuration



QC Request Scheduler ⇔ Quantum Computer Backend Server

- Quantum Computer Backend Server
 - Cloud: Existing cloud-based service. Servers are in *somewhere in the world*.
 - **Direct Access:** Faster connection via direct local connection or virtual local connection. The server for IBM will be installed in Kobe, the server for Quantinuum is installed in Wako.



Quantum-HPC Hybrid Applications

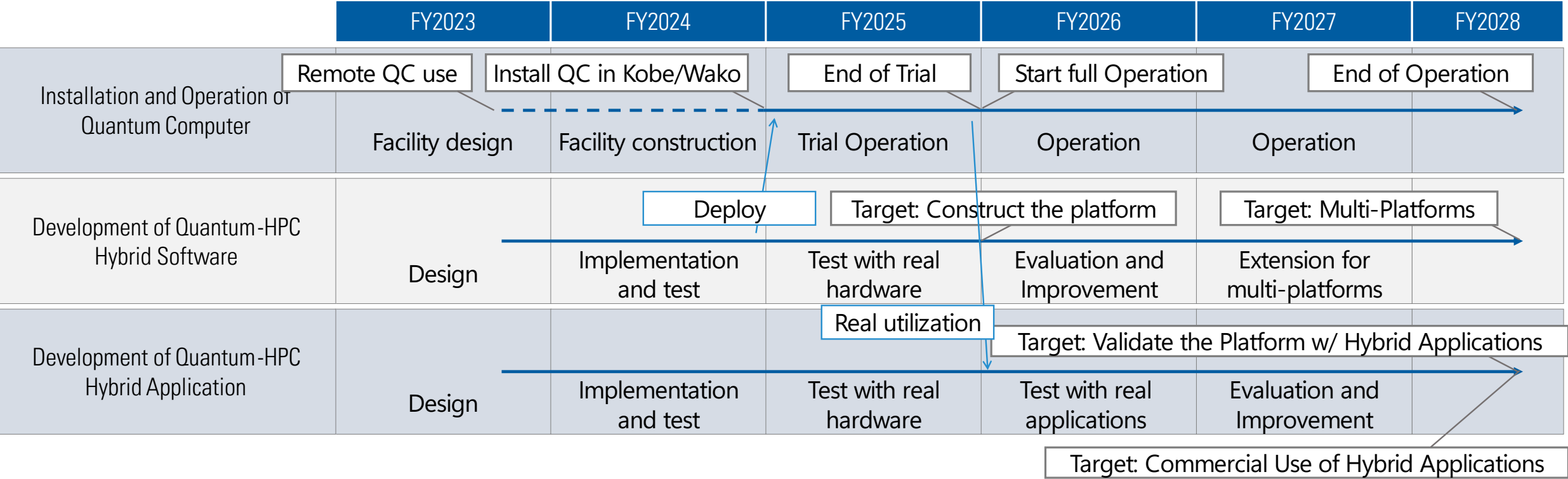
- We are focusing on the following application areas for “Utility-Scale” Quantum-HPC hybrid computing
 - Quantum Chemistry
 - Quantum Machine Learning
- Two experiences for Quantum-HPC Quantum Chemistry applications
 - Ground state energy calculation of [4Fe-4S] clusters by Sample-based quantum diagonalization (SQD)
 - Effort for Quantum-centric Supercomputing (QCSC) with IBM and RIKEN
 - SQD by Herion QPU and a large number of nodes of Fugaku (150K nodes)
 - Tightly-Coupled programming models and diagonalization by large-number of nodes.
 - Biomolecular chemical reactions by QC-HPC hybrid computing workflow
 - Effort on Biochemistry with Quantinuum and RIKEN
 - Loosely coupled programming models by Quantinuum TierKreis workflow tools
 - Photo-isomerization of retinal. Use NTChem developed by R-CCS executed in Fugaku for environment and QC for reaction centers

More Applications: Test User Program

- Call for application proposals to utilize the hybrid platform to research and develop quantum-HPC hybrid applications
- To obtain feedbacks on our hybrid platform
- 21 proposals are accepted
- 9 MOUs are concluded

User	Application area
JSR Corporation	Materials
Toyota Motor Corporation	Materials, Design & Manufacturing
SoftBank Corp.	Materials
Ochanomizu University	Natural Sciences
Toyota Central R&D Labs., Inc.	Design & Manufacturing
Kyoto University	Medical, Drug Discovery
Oita University	Medical, Drug Discovery
The University of Electro-Communications	Natural Sciences
Mitsubishi Chemical Corporation	Materials
and more ..	

Schedule



Summary

- Four Japanese Quantum HPC projects are ongoing
- Some details about JHPC-Quantum project
 - Programming model: Workflow for Loosely coupled programs and Tightly coupled tasks
 - Software stack
 - Scheduler between quantum and HPC
 - Use cases
 - Ground state energy calculation of [4Fe-4S] clusters by Sample-based quantum diagonalization
 - Biomolecular chemical reactions by quantum-HPC hybrid computing workflow
 - Test user program

Acknowledgment

- This work is supported by the Center of Innovation for Sustainable Quantum AI (JST Grant Number JPMJPF2221)
- This work is based on results obtained from a project, JPNP20017, commissioned by the New Energy and Industrial Technology Development Organization (NEDO).